Quantitative Trading: Technical & Mathematical Architecture

1. System Rules & Core Glossary

This document maps the entire quantitative trading pipeline, providing pure technical definitions, codebase locations, and explicit mathematical formulas for every mechanism.

Term	Technical Definition
t-1 Cutoff (Lookahead)	A structural barrier. To make a trade decision for candle *t*, the system may only read signal data up to the close of candle *t-1*. Implemented via `SIGNAL_LAG` shifts to prevent the model from seeing future data.
Execution at Open	Trades generated at time *t* are strictly executed at the `Open` price of candle *t*. This ensures realistic fill math compared to trading on the Close.
Transaction Friction	Cost deducted on every execution. In this project: 0.05% per side (0.10% round trip), plus slippage calculated as 5% of the candle's intraday range (High - Low) scaled by position size.
Alpha (α) / Beta (β)	**Beta:** Strategy returns generated purely by the underlying market drifting upward. **Alpha:** Excess returns generated by the timing skill of the strategy, independent of market drift.
Sharpe Ratio	A metric measuring risk-adjusted return. Calculated as the annualized average excess return divided by the annualized standard deviation of returns.
Max Drawdown	The maximum observed loss from a historical peak to a trough of a portfolio's equity curve, measured as a percentage.

Standard Performance Math `task1/performance.py`

$$\text{Sharpe Ratio} = (\mu_{returns} / \sigma_{returns}) \times \sqrt{252}$$

$$\text{Return}_t = \text{Position}_{t-1} \times [\ (\text{Open}_t / \text{Open}_{t-1}) - 1\ ] - \text{Costs}$$

These formulas calculate exactly how much money the strategy made and the volatility it suffered, adjusted to an annualized (252 trading days) figure.

2. Codebase Responsibilities

Task 1: The Backtesting Infrastructure

File Name	Role in the Pipeline
`config.py`	Stores hardcoded constants (0.05% fee, random seeds, paths).
`data_loader.py`	Loads CSVs and runs schema validation. Does not mutate data.
`data_cleaner.py`	Parses mixed date formats, removes hidden duplicate rows, and handles NaN data.
`feature_engine.py`	Constructs price-difference labels and programmatically verifies no lookahead leaks.
`strategy.py`	Abstract class defining the required `generate_signal()` interface.
`execution_engine.py`	Applies fills at the open price, deducting 0.05% fees and dynamic slippage.
`portfolio.py`	Tracks position arrays, cash balance, and mark-to-market daily equity.
`backtester.py`	Orchestrates the per-candle timeline, passing data chronologically to strategies.

Task 2: Alpha Discovery

File Name	Role in the Pipeline
`strategies/alpha_*.py`	The trading logics. 01–04 are rule-based. `alpha_05.py` is the ML logistic model.
`statistics.py`	Executes Permutation tests, Newey-West adjustments, and BH multiple-testing corrections.
`orthogonality.py`	Executes QR decomposition to prove strategy independence mathematically.

Task 3: Portfolio Allocation & Final Flow

File Name	Role in the Pipeline
`factor_model.py`	Computes Alpha/Beta decomposition via OLS regression.
`portfolio_optimizer.py`	Calculates static allocation weights (Equal, Risk Parity, Markowitz Shrinkage).
`meta_model.py`	Walk-forward ML model predicting which strategy will win the next month.
`dynamic_allocator.py`	Converts MetaModel scores to weights using Softmax and a 5% no-trade filter.
`final_evaluation.py`	Races all 5 allocation methods side-by-side on the same historical timeline.

The Task 3 Execution Flow

Task 3 acts as the final aggregator. It imports the validated, clean strategy return streams from Task 2. First, `factor_model.py` runs a mathematical check to ensure the strategies possess isolated Alpha. Next, capital is divided using four static math rules in `portfolio_optimizer.py` (like 1/N equal weight, or Markowitz shrinkage). Simultaneously, the AI `meta_model.py` performs expanding walk-forward training to actively predict strategy performance, sending target weights to `dynamic_allocator.py`. The `final_evaluation.py` script then runs all five constructed portfolios through the historical timeline to decisively benchmark their performance against transaction costs.

3. Machine Learning & AI Algorithms

Alpha05: Logistic Strategy Model

`task2/strategies/alpha_05.py`

Purpose: Predicts the direction of the 5-bar forward price change using 5 continuous signal features. Bypasses human thresholds by learning optimal feature weights.
Capacity Limit: Constrained to 6 parameters (5 weights + 1 intercept) over 600 training rows to prevent overfitting noise.

$$z = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5$$

$$P(y=1) = \sigma(z) = 1 / (1 + e^{-z})$$

$$Loss = -(1/N) \sum [y_i \log(P_i) + (1 - y_i) \log(1 - P_i)]$$

How it works: The linear combination of features (z) is squashed by the Sigmoid function (σ) into a probability. The model optimizes weights (w) by minimizing the Binary Cross-Entropy Loss over the development dataset. It goes Long (+1) if P(y=1) > 0.5.

AlphaMetaModel: Walk-Forward Ridge Logistic Allocator

`task3/meta_model.py`

Purpose: Learns per-rebalance conviction scores for each strategy to dictate dynamic portfolio weights. Uses an expanding walk-forward window to prevent lookahead bias.
Features: 63-bar trailing Sharpe ratio and 63-bar trailing hit-rate for each strategy.

$$\text{Minimize: } (1/M) \sum Loss(y_j, \sigma(w^T x_j + b)) + (1 / 2C) ||w||_2^2$$

How it works: This uses **L2 Regularization (Ridge)**. The term $||w||_2^2$ represents the sum of squared weights. Because we have very few rebalance events (~40), an unconstrained model would overfit. The penalty (with C=0.5) forces the model to keep weights small and generalized.

Softmax & No-Trade Band Filter

`task3/dynamic_allocator.py`

Purpose: Converts the MetaModel's raw conviction scores (S) into legal percentage weights, then applies a friction filter to stop excessive transaction costs.

$$w_{new,\ i} = e^{S_i} / \sum e^{S_j} \text{ (Softmax Transformation)}$$

$$w_{final,\ i} = w_{new,\ i} \text{ IF } |w_{new,\ i} - w_{old,\ i}| > 0.05 \text{ ELSE } w_{old,\ i}$$

How it works: Softmax guarantees all positive scores sum to exactly 1.0. The **No-Trade Band** mathematically blocks any portfolio adjustment unless the change in a strategy's weight exceeds 5% (0.05). If the change is small, it holds the old position.

4. Statistical Validation Math (Proving the Edge)

Permutation Test (Null-Baseline Check)

`task2/statistics.py & task3/meta_model.py`

Purpose: Tests whether a strategy or model discovered real market patterns or merely fit noise. It does this by randomly shuffling the target timeline 100+ times, destroying real correlation but preserving the distribution.

$$p\text{-value} = [\ \sum I(A_{null,\ k} \geq A_{real}) + 1\ ] \ /\ (N + 1)$$

How it works: A_{real} is the actual accuracy/return. $A_{null,\ k}$ is the accuracy from the k-th scrambled run. The indicator function $I()$ counts how many times garbage data produced a better result than the real data. If p-value > 0.10, the strategy's success is legally considered a statistical fluke.

Newey-West Autocorrelation Adjustment

`task2/statistics.py`

Purpose: Standard t-tests assume every daily return is entirely independent (i.i.d). Because trading positions are held over multiple days, returns are autocorrelated. A standard t-test will be wildly overconfident. Newey-West inflates the standard error to compensate.

$$\sigma^2_{NW} = \gamma_0 + 2 \sum_{l=1}^{L} [\ 1 - (l\ /\ (L + 1))\ ]\ \gamma_l$$

$$t\text{-stat} = \mu\ /\ (\hat{\sigma}_{NW}\ /\ \sqrt{T})$$

How it works: It adds weighted lagged covariances (γ_l) up to a lag length (L) to the base variance (γ_0). This correctly shrinks the t-stat, making it much harder for a strategy to falsely claim statistical significance.

Benjamini-Hochberg (BH) Multiple-Testing Correction

`task2/statistics.py`

Purpose: When you test 6 different alpha strategies simultaneously, the chance of finding a false positive increases dramatically. The BH procedure controls the False Discovery Rate (FDR).

$$\text{Sort p-values ascending: } p_{(1)} \leq p_{(2)} \leq \ldots \leq p_{(m)}$$

$$\text{Find largest k where: } p_{(k)} \leq (k\ /\ m) \times \alpha_{FDR}$$

How it works: m is the number of strategies tested (6). α_{FDR} is the target false discovery rate (e.g., 0.10). The required threshold for significance becomes exponentially stricter for the most significant strategies. Even if Alpha02 achieves a raw p=0.096, it fails BH adjustment because it doesn't clear the lowered bar dictated by testing 6 ideas at once.

5. Portfolio & Return Redundancy Math

Factor Model (Alpha vs Beta OLS)

`task3/factor_model.py`

Purpose: Ordinary Least Squares regression isolates true timing skill from market drift.

$$r_{strategy}(t) = \alpha + \beta \times F_{market}(t) + \varepsilon(t)$$

$$\beta = Cov(r_{strategy}, F_{market}) \;/\; Var(F_{market})$$

How it works: The asset's buy-and-hold return is the independent variable `F(t)`. The strategy's return is `r(t)`. Beta (β) represents the linear correlation to the market. Alpha (α) is the y-intercept: the residual, independent excess return that the strategy uniquely generated.

Orthogonality Analysis (Gram-Schmidt & QR Decomposition)

`task2/orthogonality.py`

Purpose: Proves if multiple strategies are genuinely distinct. If Strategy A and Strategy B make the exact same trades, B is mathematically redundant. We use **Gram-Schmidt Orthogonalization** to mathematically subtract the copied parts so we can isolate the pure new edge.

$$Proj_A(B) = [\ (B \cdot A) \;/\; (A \cdot A)\ ] \times A$$

$$Pure_B = B - Proj_A(B)$$

How Gram-Schmidt works: Think of the daily returns of Strategy A and Strategy B as two separate lines (vectors).
• **The Dot Product (B · A):** Calculates exactly how much Strategy B's profit overlaps and moves in the exact same direction as Strategy A.
• **The Projection (Proj):** Creates a "shadow" of Strategy B that represents the redundant overlapping returns.
• **The Subtraction:** We subtract that overlapping shadow from B. The leftover (`Pure_B`) is the 100% unique profit. If `Pure_B` equals zero, Strategy B is entirely a clone.

$$\text{Matrix QR Version: } R = Q \times R_{upper} \times P^T$$

How QR Decomposition works: This is the Gram-Schmidt process applied to *all strategies at once*. The matrix of all strategy returns R is factored into a matrix of pure, unique directions Q and a scaling matrix R_{upper} (using column pivoting P). If any diagonal number in R_{upper} drops to zero, it instantly flags that a newly added strategy is a mathematical clone of the others combined.

Markowitz Mean-Variance Shrinkage

`task3/portfolio_optimizer.py`

Purpose: Finds the mathematically optimal blend of strategies to maximize portfolio Sharpe ratio. Addresses the instability of small-sample covariance matrices.

$$\text{Raw Weights: } w = \Sigma_{shrunk}^{-1} \times \mu$$

$$\Sigma_{shrunk} = (1 - \lambda)\ \Sigma_{sample} + \lambda\ \Sigma_{target}$$

How it works: μ is the vector of expected returns. Σ_{sample} is the observed covariance matrix. Because 1000 days is a small sample, inverting Σ_{sample} creates wild, extreme allocations. We "shrink" the matrix by blending it (λ = 0.3) with a safe target matrix Σ_{target} (where all off-diagonal correlations are zero). This stabilizes the math, allowing safe calculation of the inverted covariance matrix Σ^{-1} to generate stable optimal weights.

6. Project Walkthrough: Step-by-Step Tutorial

This section teaches you how the entire Multi-Alpha Lab is executed chronologically. If you were handed this project today, here is exactly how you would build it from the ground up.

Phase 1: Building the Engine Room (The Sandbox)

The Goal: Before you can invent a trading strategy, you must build a flawless simulation engine. If your engine is broken, every test result is a lie.

Step 1 (Data Prep): You receive raw CSV files containing prices and 20 anonymized signals. You pass them through `data_loader.py` and `data_cleaner.py`. Real data is dirty—you must write code to fix European vs. US date formats, delete hidden duplicate rows, and leave rolling calculation "warm-up" periods as blank (NaN) rather than faking data.

Step 2 (The Backtester): You build `backtester.py` and `execution_engine.py`. You enforce the absolute golden rule of quant finance: the **t-1 cutoff**. You write code that physically shifts the signal data forward by one row. This guarantees that when your strategy asks "what should I do on Tuesday?", it physically cannot access Wednesday's data. Finally, you program the execution engine to deduct a 0.05% broker fee on every single trade, executed strictly at the opening price.

Result: You now have a mathematically honest sandbox. Any strategy that survives this sandbox might actually survive the real world.

Phase 2: The Alpha Hunt (Strategy Research)

The Goal: Invent trading rules using the 20 signals and test if they have genuine predictive power (Alpha).

Step 1 (Idea Generation): You look at the 20 signals (Price, Band, and Volume data) and write five distinct strategies. For example, *Alpha02* buys when prices hit the bottom of a Bollinger Band (expecting a bounce). *Alpha04* buys when market volume suddenly spikes after a quiet period. *Alpha05* is your ML strategy: it uses Logistic Regression to learn the patterns of 5 continuous signals.

Step 2 (The Execution): You plug all five strategies into your Phase 1 Sandbox. The Sandbox spits out a daily equity curve (profit/loss timeline) for each strategy.

Step 3 (The Interrogation): Now, you must act as your own harshest critic. You run `statistics.py`. You use Permutation Tests to shuffle the timeline—if the strategy still makes money on random data, you throw it out. You use the BH Correction because you tested 5 ideas at once (which increases the chance of a lucky fluke). You use `orthogonality.py` to ensure Alpha02 and Alpha04 aren't just making the exact same trades.

> **The Harsh Reality of Phase 2**
>
> In this specific dataset, **all five strategies failed the final statistical interrogation**. None of them produced an edge strong enough to statistically prove it wasn't just luck after accounting for trading costs. This "Null Result" is a successful scientific finding. However, to complete Phase 3, we carry forward the top three unvalidated strategies anyway.

Phase 3: Building the Empire (Portfolio Construction)

The Goal: You have a pool of capital and three surviving strategies. You must build a system to mathematically decide how to divide your money among them every day.

Step 1 (The Alpha/Beta Check): You run `factor_model.py`. This runs an OLS regression to ensure your three strategies aren't just "long the market" (Beta). You confirm that they are at least attempting to time the market (Alpha), meaning they are worth combining.

Step 2 (Static Baselines): You build `portfolio_optimizer.py` to test traditional math. You write an "Equal Weight" allocator (33% to each). You write a "Markowitz Shrinkage" allocator, which looks at the covariance matrix (how the strategies zig and zag together) and finds the mathematically safest blend of the three.

Step 3 (The AI Manager): You build the `AlphaMetaModel`. Instead of a static 33% split forever, this ML model looks at the trailing 3-month performance of the strategies. If Strategy A is on a winning streak, the AI predicts it will win next month. It sends these predictions to the `dynamic_allocator.py`. To protect your capital, you add the **5% No-Trade Band**—if the AI only wants to shift capital by 2%, the allocator ignores it to save on broker fees.

Step 4 (The Grand Finale): You run `final_evaluation.py`. You put the Equal Weight portfolio, the Markowitz portfolio, and the AI Manager portfolio on the same starting line and race them through the 4-year backtest engine.

> **The Final Conclusion of the Lab**
>
> When the race finishes, the **Markowitz Shrinkage (Static Math)** wins with a Sharpe Ratio of 0.189. The advanced AI MetaModel fails to beat the simple equal-weight baseline (Sharpe 0.009). Furthermore, the permutation test on the AI proves its predictions were no better than random guessing (p=0.20).

> **The Ultimate Lesson for a Beginner**
>
> This project teaches that quantitative trading is not about throwing complex AI at market data. It is about **extreme discipline**. By enforcing strict rules (t-1 cutoffs, transaction fees, multiple-testing corrections, and null-baselines), you prevent yourself from being fooled by random noise. Complexity (like the MetaModel) often fails where robust, simple math (like Covariance Shrinkage) succeeds.